

Carlos Elizondo
500890062

Part I: Relational Engineering and Schema Validation

Task 1:

The Database Model is attached with the name Carlos_Elizondo_A1_P1_T1

Task 2:

The Schema is attached with the name Carlos_Elizondo_A1_P1_T2

Task 3:

Establishing one-to-many and non-identifying relationships ensures operational stability and maintains a source of truth during national scaling. In terms of operational stability, the dashlines connecting the primary key to the foreign key allow a child record to exist without being part of the primary identity of its parent. This flexibility allows records like orders or inventory to be added without making the database too rigid. It allows the entities to stay independent while still being linked, so if they add more rows across the country the system stays stable.

The source of truth applies to national scaling as these relationships centralize data. By linking FamilyId in the families and flowers tables the information about the flower family is only stored in the families table and is accessed by the flowers table. This helps prevent data duplication and that all the stores nationally will have the same information. If they update the family name, it will be updated across all stores nationally automatically.

By having a foreign key constraint, if a shop is entering the wrong flower id, such as Flower ID:60, the entry will be blocked as it is not part of the known values since the Flower Id only goes up to 40. This is beneficial when looking at what flowers sell the most so that there are no amounts for flowers that are not in the database.

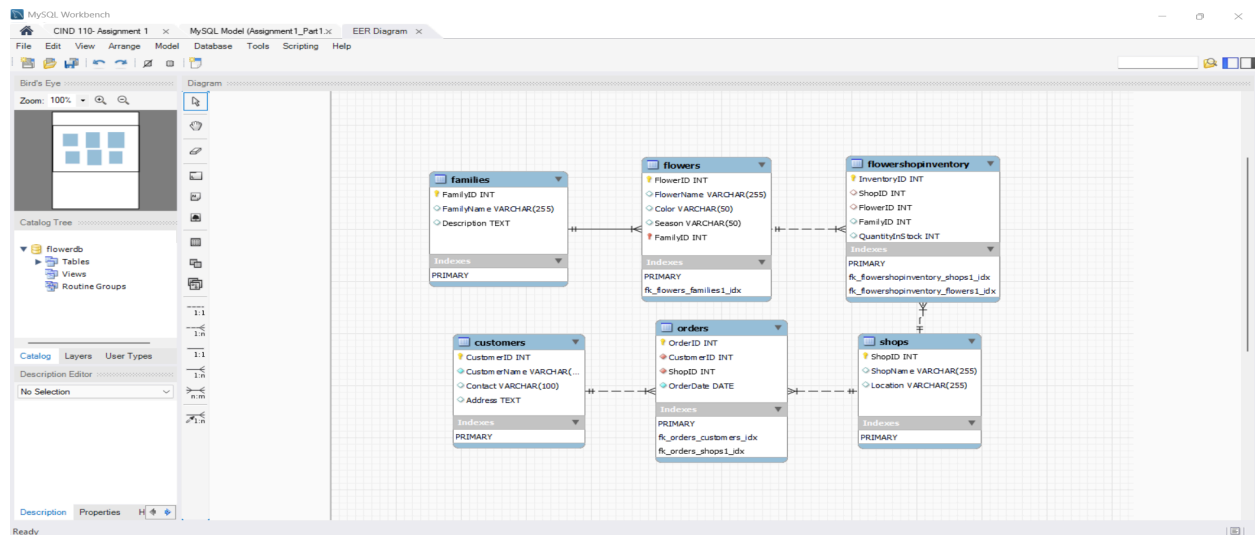


Figure 1: EER Diagram

Part II: SQL Analysis and Business Intelligence

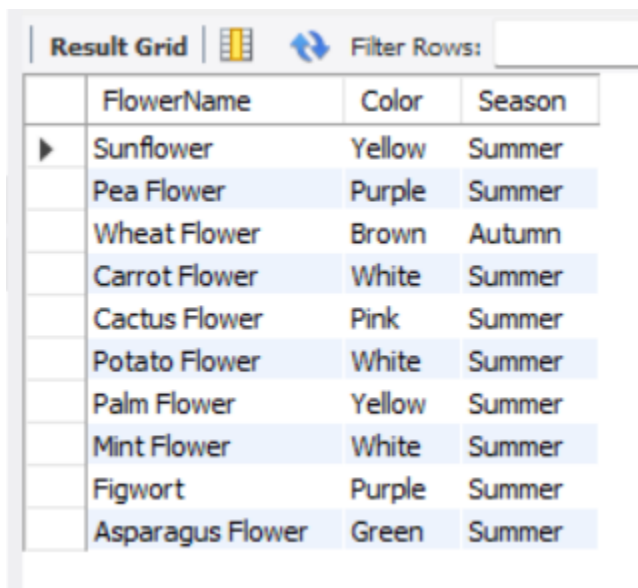
1. Consumer Behavior (Attribute Filtering):

GOAL:

Find flowers that are not Red and are available outside the Winter/Spring seasons.

CODE:

```
USE flowerdb;  
SELECT FlowerName, Color, Season  
FROM flowers  
WHERE Color <> 'Red'  
AND Season NOT IN ('Spring', 'Winter')  
LIMIT 10;
```



The screenshot shows a 'Result Grid' interface with a 'Filter Rows' input field. Below the header, there are 10 rows of data. The first row is highlighted with a blue background. The columns are FlowerName, Color, and Season.

FlowerName	Color	Season
Sunflower	Yellow	Summer
Pea Flower	Purple	Summer
Wheat Flower	Brown	Autumn
Carrot Flower	White	Summer
Cactus Flower	Pink	Summer
Potato Flower	White	Summer
Palm Flower	Yellow	Summer
Mint Flower	White	Summer
Figwort	Purple	Summer
Asparagus Flower	Green	Summer

Figure 2: Flowers that are not red and are available in the Summer and Autumn

DIAGNOSIS:

Red flowers, like roses, are sold during the winter season such as on valentine's day. The reason to find what flowers aren't red and aren't part of spring and winter can help the company to know what they should be looking at selling during their off season and give their clients other colour flowers. As the company plans their off season the flowers in Figure 2 are the ones the company can market to sell during the Summer and Autumn season. This will help the company know what they will be selling and have their inventory match.

STRATEGIZE:

Flowers like Sunflowers, Cactus flowers, and pea flowers are found in the summer and the company can use them in their advertisements for the summer season. They can also provide a competitive price during the summer season as an incentive for the customers to buy that kind of flower.

2. Inventory Gaps (Relationship Analysis):

GOAL:

Identify flowers that are not in stock at any shop location

CODE:

```
USE flowerdb;
```

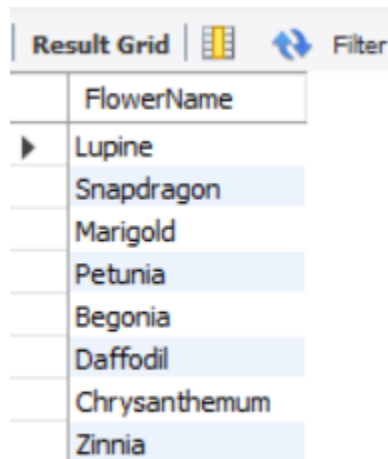
```
SELECT flowers.FlowerName
```

```
FROM flowers
```

```
LEFT JOIN flowershopinventory ON flowers.FlowerID = flowershopinventory.FlowerID
```

```
WHERE flowershopinventory.InventoryID IS NULL
```

```
LIMIT 10;
```



The screenshot shows a database interface with a 'Result Grid' tab selected. The grid displays a list of flower names. The first row is 'Lupine'. The subsequent rows are 'Snapdragon', 'Marigold', 'Petunia', 'Begonia', 'Daffodil', 'Chrysanthemum', and 'Zinnia'. Each row has a small blue arrow icon to its left, indicating it can be expanded or collapsed. The grid is titled 'FlowerName'.

FlowerName
Lupine
Snapdragon
Marigold
Petunia
Begonia
Daffodil
Chrysanthemum
Zinnia

Figure 3: Flowers not in Inventory

DIAGNOSIS:

These flowers are in our database but are not part of the inventory of any of the stores at the moment. These are a potential missed opportunity by the current stores. Since the company has them categorized in the database already, they are missing potential customers by not having them stocked.

STRATEGIZE:

The company should check how popular these flowers were in the past. If the flowers were profitable they need to investigate why they no longer have them at the stores. If the flowers were popular in the past they could do a promotion in the stores with a limited amount of Petunias to start to check if there is a demand for them. Otherwise they could also remove the flowers from their catalog.

3. Client Engagement (Multi-Location Loyalty):

GOAL:

List customers who have shopped at multiple locations.

CODE:

```
USE flowerdb;
```

```
SELECT customers.CustomerName, COUNT(DISTINCT orders.ShopID)
```

```

FROM customers
JOIN orders ON customers.CustomerID = orders.CustomerID
GROUP BY customers.CustomerName
HAVING COUNT(DISTINCT orders.ShopID) >= 2
LIMIT 10;

```

	CustomerName	COUNT(DISTINCT orders.ShopID)
▶	Alice Smith	2
	Diana Howard	3
	Fiona Edwards	5
	Noah Adams	2
	Victor Lee	4

Figure 4: Customers who have shopped at two or more locations.

DIAGNOSIS:

This query is identifying customers who have shopped at multiple locations. These customers have chosen to buy flowers from this company at various locations, demonstrating that they have loyalty to this company and not that it is their local flower shop. Fiona Edwards has shopped at 5 different stores with this flower company.

STRATEGIZE:

The CEO can set up email surveys to be sent to customers who have shopped at least at 2 stores to better understand why they like shopping at this company. Knowing what part of the service or product attracts customers to keep shopping with them can strengthen the customer service. Keep in mind that another query is needed to see the quantity that these customers spend. There could be loyal customers who have multiple orders at the same shop. Those stores should also be surveying their customers to know why they continue to shop with them.

4. Stock Volume Optimization (Nested Aggregation):

GOAL:

Find Flower families that maintain an average stock volume of 140 units or more.

CODE:

```

USE flowerdb;
SELECT FamilyID, AVG(TotalInShop) AS AverageStockPerShop
FROM (SELECT FamilyID, ShopID, SUM(QuantityInStock) AS TotalInShop FROM
flowershopinventory GROUP BY FamilyID, ShopID)
AS ShopLevelAggregation
GROUP BY FamilyID
HAVING AverageStockPerShop >= 140
LIMIT 10;

```

	FamilyID	AverageStockPerShop
►	19	140.0000
	25	140.0000
	26	145.0000
	27	150.0000
	28	160.0000
	30	160.0000

Figure 5: Flower families with high inventory

DIAGNOSIS:

This query is to identify the flower families that have a high inventory across all stores. Those flower families have an average stock of 140 or more which means they are making up a large portion of the physical inventory.

STRATEGIZE:

The CEO should now determine if these flower families are also selling a lot, meaning it is optimal to keep them in inventory as they move quickly. If the flower families aren't selling as quickly they could reduce the amount they are buying to avoid having too many in stock and allocate the money to get more popular flowers. The results of further research can determine whether or not to offer a discount for those flowers to move inventory and reduce the amount left over in stock.

5. Operational Risk Monitoring (Group Constraints):

GOAL:

Identify shops where every single flower in stock is over 100 units

CODE:

USE flowerdb;

SELECT ShopID, MIN(QuantityInStock) AS SmallestItemCount

FROM flowershopinventory

GROUP BY ShopID

HAVING SmallestItemCount > 100

LIMIT 10;

	ShopID	SmallestItemCount
►	10	105
	11	115
	12	125
	13	135
	14	145
	15	155
	16	165

Figure 6: shop locations where every individual inventory quantity exceeds 100 units

DIAGNOSIS:

This query is finding the shops where the flower they have the least of has more than 100 in stock. The list identifies shops that are overstocked and if sales drop in those stores they are at a higher risk of having inventory expire due to having too many.

STRATEGIZE:

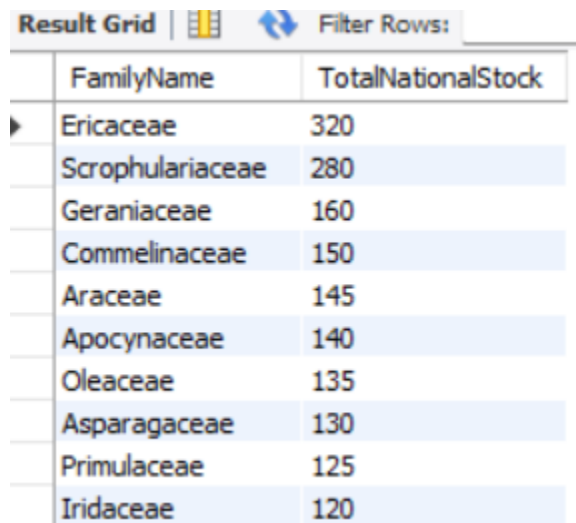
The CEO can look at the sales volume for these stores to see if they are ordering too many flowers. This is also an opportunity to move the overstock to stores that are maybe low on a certain flower, rather than having to order more flowers from suppliers.

6. Enterprise Stock Summary (Broad Aggregation):**GOAL:**

The total stock per flower family across the entire organization.

CODE:

```
USE flowerdb;
SELECT families.FamilyName, SUM(flowershopinventory.QuantityInStock) AS
TotalNationalStock
FROM families
JOIN flowershopinventory ON families.FamilyID = flowershopinventory.FamilyID
GROUP BY families.FamilyName
ORDER BY TotalNationalStock DESC
LIMIT 10;
```



The screenshot shows a database interface with a 'Result Grid' tab selected. The grid displays the results of a SQL query, showing the top 10 flower families by total stock. The columns are 'FamilyName' and 'TotalNationalStock'. The rows are listed in descending order of stock quantity.

FamilyName	TotalNationalStock
Ericaceae	320
Scrophulariaceae	280
Geraniaceae	160
Commelinaceae	150
Araceae	145
Apocynaceae	140
Oleaceae	135
Asparagaceae	130
Primulaceae	125
Iridaceae	120

Figure 7: Total Stock per Flower Family

DIAGNOSIS:

This Query shows the stock per flower family in descending order. This shows how much stock the company has across all stores and these are the top 10 flower families in terms of current stock.

STRATEGIZE:

It is important to know what the total inventory is composed of. This helps us determine what flowers we need to focus on selling, and can run promotions if they are not selling fast enough to prevent the flowers from wilting.

7. Proprietary Diagnostic Query:

How many different individual flower species do we have within each flower family?

GOAL:

To determine how many individual flowers are part of the same flower family

CODE:

```
USE flowerdb;
```

```
SELECT families.FamilyName, COUNT(flowers.FlowerID) AS SpeciesCount
```

```
FROM families
```

```
JOIN flowers ON families.FamilyID = flowers.FamilyID
```

```
GROUP BY families.FamilyName
```

```
HAVING SpeciesCount > 1
```

```
ORDER BY SpeciesCount DESC;
```

	FamilyName	SpeciesCount
►	Asteraceae	3
	Rosaceae	2
	Fabaceae	2
	Brassicaceae	2
	Solanaceae	2
	Myrtaceae	2
	Arecaceae	2
	Caryophyllaceae	2
	Scrophulariaceae	2
	Ericaceae	2

Figure 8: Individual flowers per flower family

DIAGNOSIS:

This query identifies the flower families that offer the most variety in the company's catalog. The Asteraceae flower family has three different species. This can help determine what flower families employees should know more about to help with questions customers may have.

STRATEGIZE:

This query can help to see areas of opportunity there are for the company to grow their catalog. They currently have 3 types of daisies but only one type of orchid. The company can offer different types of flowers from the same flower family to have a competitive advantage when launching into the national market.

Part III: Critical Retrospective and Professional Growth

Phase 1: Technical Retrospective (Max 100 words)

For this project, I turned a flower shop's data into a cloud-style database. I connected the tables from the flower shop data with one to many and non-identifying relationships. Afterwards the assignment required SQL coding to find specific groups of data, like totaling up stock or filtering for certain seasons and colours. A challenge I faced was in Requirement 4. The query needed to first find the total for each shop before I could find the national average. I had to write a nested query and run one piece at a time.

Phase 2: Strategic Alignment (Max 200 words)

The SQL code helped to use data when making business decisions rather than making those decisions intuitively. Requirement one gave flowers that aren't red and grow in the summer and fall. This can help the company know what flowers they should focus on selling during those seasons and adjust their marketing strategies accordingly. As the company is getting ready for the national expansion, they need to know what flower is in season to have a uniformed approach as a company across Canada.

The data also showed what flower families the company has in inventory. However, one problem I found with the database is that it is missing the value in dollars of each order. Knowing how much each flower is bringing in Revenue can signal the underperforming flowers and the best sellers. That is important because it will help the company decide what flowers they should have more in their inventory and which ones can be removed from their catalog. It can be expensive to carry every type of flower every season. Another limitation is the lack of Province in the customer address so it makes it harder to pinpoint the best location for the next store.

Phase 3: Synthesis and Transferability (Max 200 words)

To make this better for the future, I would suggest adding a Price column and a Revenue column. This would let the company know which flowers are the most profitable and which are underperforming. This can help with inventory and also to know when to make a strategic sale to move inventory and prevent the flowers from wilting.

This project taught me that a data analyst isn't just someone who writes code; the data and tables that are created solve a specific question, like spotting which shops have too much stock sitting on the shelves and provide suggestions to solve the problem. The skills I practiced, like joining different tables together and grouping data to find totals, are things I can use in any job that uses data to solve problems.